Bab 7

Praktikum 6: Searching, Sorting, dan Benchmark

Pencarian: Mencari ada atau tidak
Sorting mengurutkan: acak -> sorted

Pencarian yang butuh data dalam kondisi sorted

Tujuan praktikum

Mahasiswa mampu menelusuri proses searching dan sorting langkah demi langkah, membedakan arti satu *pass* pada beberapa algoritma sorting, mengimplementasikan selection sort, bubble sort, dan merge sort, menggunakan hasil sorting sebagai prasyarat binary search, serta menghubungkan pengukuran `timeit` dengan analisis kompleksitas algoritma.

7.1 Searching pada data terurut dan tidak terurut

Sequential search memeriksa elemen satu per satu sehingga dapat digunakan pada data yang belum terurut. Binary search membutuhkan data terurut karena pada setiap iterasi algoritma membuang separuh daerah pencarian.

```
def sequential_search(a, target):
    for i in range(len(a)):
        if a[i] == target:
            return i
    return -1


def binary_search(a, target):
    kiri, kanan = 0, len(a) - 1
    while kiri <= kanan:
        tengah = (kiri + kanan) // 2
        if a[tengah] == target:
            return tengah
        if a[tengah] < target:
            kiri = tengah + 1
        else:
            kanan = tengah - 1
    return -1
```

7.1.1 Tracing binary search

Untuk list terurut `[3,5,6,8,11,12,14,15,17,18]`, catat pada setiap iterasi: indeks kiri, indeks kanan, indeks tengah, nilai tengah, dan keputusan algoritma. Jika target lebih kecil dari nilai tengah, interval kanan dibuang; jika target lebih besar, interval kiri dibuang.

```
def binary_search_trace(a, target):
```

```
    kiri, kanan = 0, len(a) - 1
    langkah = 1

    while kiri <= kanan:
        tengah = (kiri + kanan) // 2
        print("langkah", langkah,
              "kiri=", kiri,
              "tengah=", tengah,
              "kanan=", kanan,
              "nilai=", a[tengah])

        if a[tengah] == target:
            print("target ditemukan")
            return tengah
        elif a[tengah] < target:
            kiri = tengah + 1
        else:
            kanan = tengah - 1

        langkah += 1

    print("target tidak ditemukan")
    return -1
```

7.2 Selection sort: arti satu *pass*

Pada selection sort menaik, satu *pass* berarti:

1. tentukan posisi pertama pada bagian yang belum terurut;
2. cari nilai minimum pada bagian tersebut;
3. tukar minimum dengan elemen pada posisi pertama bagian yang belum terurut;
4. setelah *pass* ke-k, sebanyak k elemen terkecil sudah berada pada posisi final di sebelah kiri.

Contoh awal:

$$[11, 7, 12, 14, 19, 1, 6, 18, 8, 20].$$

Tiga *pass* pertama adalah

pass	minimum yang dipilih	keadaan list setelah pass
1	1	$[1, 7, 12, 14, 19, 11, 6, 18, 8, 20]$
2	6	$[1, 6, 12, 14, 19, 11, 7, 18, 8, 20]$
3	7	$[1, 6, 7, 14, 19, 11, 12, 18, 8, 20]$

7.2.1 Kode yang mencetak setiap proses pengulangan

Kode berikut sengaja verbose untuk praktikum. Setiap perbandingan pada loop dalam dicetak sehingga mahasiswa dapat melihat bagaimana kandidat minimum berubah, lalu melihat hasil akhir satu *pass*.

```
def selection_sort_trace(data):
    a = data.copy()
    n = len(a)

    for i in range(n - 1):
        idx_min = i
```

```
        print(f"\nPASS {i+1}")
        print("awal :", a)

        for j in range(i + 1, n):
            print(f"bandingkan a[{j}]={a[j]} "
                  f"dengan minimum sementara "
                  f"a[{idx_min}]={a[idx_min]}")

            if a[j] < a[idx_min]:
                idx_min = j
                print("  -> minimum baru:", a[idx_min])

        a[i], a[idx_min] = a[idx_min], a[i]
        print("swap  :", a)
        print("final :", a[:i+1],
              "| belum terurut:", a[i+1:])

    return a
```

Catatan

Untuk memahami algoritma, gunakan versi `*_trace`. Untuk benchmark, gunakan versi tanpa `print`, karena operasi pencetakan jauh lebih mahal daripada perbandingan dan pertukaran yang sedang dianalisis.

7.3 Bubble sort: setiap perbandingan dapat terlihat

Pada bubble sort, satu *pass* adalah satu lintasan dari kiri ke kanan yang membandingkan pasangan bersebelahan. Jika urutannya salah, kedua elemen ditukar. Setelah satu *pass* penuh, elemen terbesar pada bagian aktif berada di posisi paling kanan bagian tersebut.

```
def bubble_sort_trace(data):
    a = data.copy()
    n = len(a)

    for p in range(n - 1):
        print(f"\nPASS {p+1}")

        for j in range(0, n - 1 - p):
            print(f"bandingkan {a[j]} dan {a[j+1]}")

            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                print("  swap ->", a)
            else:
                print("  tetap ->", a)

        print("hasil pass:", a)

    return a
```

Perbedaan arti *pass*

Algoritma	Makna satu langkah utama
Selection sort	satu *pass*: mencari minimum pada bagian belum terurut, lalu satu pertukaran ke posisi final di kiri
Bubble sort	satu *pass*: satu lintasan pasangan bersebelahan; elemen terbesar bergerak ke kanan
Merge sort	lebih tepat ditelusuri sebagai level pembagian dan tahap penggabungan, bukan *pass* seperti selection/bubble

7.4 Merge sort: *divide, conquer, merge*

Merge sort tidak mencari minimum atau menukar pasangan bersebelahan. Algoritma bekerja dengan dua tahap besar.

7.4.1 Tahap 1: membagi masalah

Untuk list

$$[21, 1, 26, 45, 29, 28, 2, 9],$$

algoritma membelah list menjadi dua sampai setiap sublist berukuran satu:

$$\begin{aligned}&[21, 1, 26, 45, 29, 28, 2, 9]\\ &\quad\to [21, 1, 26, 45] \mid [29, 28, 2, 9]\\ &\quad\to [21, 1] \mid [26, 45] \mid [29, 28] \mid [2, 9]\\ &\quad\to [21]||[1]||[26]||[45]||[29]||[28]||[2]||[9].\end{aligned}$$

Sublist satu elemen sudah terurut secara trivial. Ini adalah *base case* rekursi.

7.4.2 Tahap 2: menggabungkan dua list terurut

Fungsi `merge(left,right)` selalu menerima dua list yang **sudah terurut**. Bandingkan elemen paling depan; pindahkan yang lebih kecil ke hasil; ulangi sampai salah satu list habis; lalu tambahkan sisa list lainnya.

Contoh:

$$[1, 21] + [26, 45] \longrightarrow [1, 21, 26, 45].$$

```
def merge(left, right, trace=False):
    hasil = []
    i = j = 0

    while i < len(left) and j < len(right):
        if trace:
            print("bandingkan", left[i], "dan", right[j])

        if left[i] <= right[j]:
            hasil.append(left[i])
            i += 1
        else:
            hasil.append(right[j])
            j += 1
```

```
15
16          if trace:
17              print("hasil sementara:", hasil)
18
19      hasil.extend(left[i:])
20      hasil.extend(right[j:])
21      return hasil
```

7.4.3 Algoritma rekursif merge sort

1. Jika panjang list ≤ 1, kembalikan list tersebut.
2. Tentukan indeks tengah.
3. Urutkan bagian kiri secara rekursif.
4. Urutkan bagian kanan secara rekursif.
5. Gabungkan dua hasil terurut dengan `merge`.

```
1  def merge_sort(data, depth=0, trace=False):
2      a = data.copy()
3
4      if len(a) <= 1:
5          if trace:
6              print("  " * depth, "base:", a)
7          return a
8
9      mid = len(a) // 2
10
11     if trace:
12         print("  " * depth, "split:", a,
13               "->", a[:mid], "|", a[mid:])
14
15     left = merge_sort(a[:mid], depth + 1, trace)
16     right = merge_sort(a[mid:], depth + 1, trace)
17     hasil = merge(left, right, trace=trace)
18
19     if trace:
20         print("  " * depth, "merge:",
21               left, "+", right, "->", hasil)
22
23     return hasil
```

Mengapa $O(n \log n)$?

Pembagian menghasilkan sekitar $\log_2 n$ level karena ukuran list terus dibagi dua. Pada setiap level, total elemen yang diproses saat proses *merge* tetap sekitar n. Maka

$$T(n) \approx n \log_2 n, \qquad O(n \log n).$$

7.5 Sorting sebagai tahap sebelum binary search

Binary search hanya aman digunakan jika data sudah terurut. Karena itu, sebuah prosedur dapat memiliki dua tahap: **sorting terlebih dahulu**, kemudian **searching**. Untuk satu kali pencarian,

biaya sorting dapat lebih besar daripada sequential search; tetapi jika data yang sama akan dicari berkali-kali, biaya sorting dapat menjadi investasi awal yang berguna.

Gunakan data berikut:

$$D = [11, 7, 12, 14, 19, 1, 6, 18, 8, 20].$$

Urutkan D dengan salah satu algoritma yang telah dibuat, kemudian gunakan binary search untuk mencari 14 dan 9. Catat proses sorting dan proses searching secara terpisah.

7.6 Benchmark sorting dengan `timeit`

Versi trace tidak digunakan untuk benchmark. Buat versi `selection_sort`, `bubble_sort`, dan `merge_sort` tanpa `print`. Gunakan data awal yang sama pada setiap algoritma.

```
import random
import timeit
from math import log2


def buat_data(n, seed=42):
    rng = random.Random(seed)
    return [rng.randint(0, 10**6) for _ in range(n)]


def ukur(sort_func, data):
    # copy agar setiap algoritma menerima kondisi awal yang sama
    hasil = timeit.repeat(
        lambda: sort_func(data.copy()),
        repeat=3,
        number=1
    )
    return min(hasil)
```

Catatan

Jangan mencoba membuat list Python berukuran 10^{12} atau 10^{18}. Bahkan 10^8 integer dapat membutuhkan memori yang sangat besar. Ukuran ekstrem pada latihan digunakan untuk menilai apakah mahasiswa dapat membedakan **benchmark aktual** dan **prediksi berdasarkan model kompleksitas**.

Jika waktu aktual untuk $n_0 = 1000$ adalah t_0, gunakan model pertumbuhan untuk memperoleh estimasi kasar:

$$\hat{t}(n) = t_0 \left(\frac{n}{n_0} \right)^2$$

untuk selection/bubble sort, dan

$$\hat{t}(n) = t_0 \frac{n \log_2 n}{n_0 \log_2 n_0}$$

untuk merge sort.

```
def estimasi_n2(t0, n0, n):
    return t0 * (n / n0)**2


def estimasi_nlogn(t0, n0, n):
    return t0 * ((n * log2(n)) /
                 (n0 * log2(n0)))
```

Latihan Praktikum 6

Latihan nomor 1, 2, 3 (kalau merge sort ok), 4 (hitung waktu)

1. Jalankan `selection_sort_trace` pada `[11,7,12,14,19,1,6,18,8,20]`. Untuk tiga *pass* pertama, catat setiap kandidat minimum, posisi pertukaran, dan keadaan list setelah pertukaran.

2. Jalankan `bubble_sort_trace` pada `[11,7,12,14,19,1]`. Jelaskan perbedaan antara satu perbandingan, satu pertukaran, dan satu *pass* penuh.

3. Untuk `[21,1,26,45,29,28,2,9]`, tulis secara manual seluruh pembagian sampai *base case*, lalu lakukan penggabungan kembali sampai list terurut. Setelah itu cocokkan dengan keluaran `merge_sort(..., trace=True)`.

4. Gunakan data
$$D = [11, 7, 12, 14, 19, 1, 6, 18, 8, 20].$$
Urutkan D dengan selection sort dan merge sort. Pastikan kedua hasil sama. Gunakan hasil yang sudah terurut untuk binary search terhadap target 14 dan 9. Jelaskan mengapa binary search tidak boleh langsung diterapkan pada D sebelum sorting.

Tugas: 5-10

5. Diberikan dictionary berikut.

```
mahasiswa = {
    "Nadia Prameswari": "125160047",
    "Raka Mahendra":    "125160012",
    "Salsa Nirmala":    "125160083",
    "Dimas Ardiansyah": "125160025",
    "Keisha Ramadhani": "125160061",
    "Farel Wicaksana":  "125160008",
    "Tania Maharani":   "125160074",
    "Bagas Pratama":    "125160039"
}
```

Rancang fungsi `urutkan_berdasarkan_nim(data)` yang mengembalikan **dictionary baru** dengan urutan entri berdasarkan NIM dari kecil ke besar. Untuk versi utama, jangan gunakan `sorted`; gunakan salah satu algoritma sorting yang dipelajari terhadap pasangan `(nama, nim)`. Jelaskan mengapa dictionary hasil tetap dapat mempertahankan urutan penyisipan pada Python modern.

6. Dengan dictionary yang sama, rancang `urutkan_berdasarkan_nama(data)` yang mengembalikan dictionary baru dengan nama terurut alfabetis. Bandingkan bagian algoritma yang berubah dengan pengurutan berdasarkan NIM.

7. Setelah dictionary diurutkan berdasarkan NIM, buat fungsi pencarian NIM tertentu yang mengembalikan nama mahasiswa. Pilih apakah sequential search atau binary search lebih tepat terhadap representasi data yang Anda buat, lalu jelaskan prasyaratnya.

8. Buat bilangan bulat acak dan ukur waktu sorting dengan `timeit`. Gunakan `n=1000` untuk benchmark aktual selection sort, bubble sort, dan merge sort. Gunakan pola berikut sebagai petunjuk:

```
data = buat_data(1000, seed=42)
t = ukur(selection_sort, data)
print(t)
```

Ulangi untuk fungsi sorting lainnya dengan **data awal yang sama**.

9. Pertimbangkan ukuran
$$n = 10^3, \quad 10^8, \quad 10^{12}, \quad 10^{18}.$$

No 9 tidak perlu pusing dengan feasible.

Untuk 10^3, isi waktu aktual dari `timeit`. Untuk ukuran yang tidak feasible dibuat di memori atau dijalankan sampai selesai, tulis *tidak feasible* pada kolom waktu aktual dan hitung **estimasi waktu** dari model $O(n^2)$ atau $O(n \log n)$ menggunakan fungsi `estimasi_n2` dan `estimasi_nlogn`.

10. Sajikan hasil nomor 8–9 dalam tabel berikut dan interpretasikan perbedaannya.

n	selection	bubble	merge	keterangan
10^3				
10^8				
10^{12}				
10^{18}				

Bedakan dengan jelas nilai hasil `timeit` dan nilai hasil estimasi matematis.

11. Untuk masing-masing algoritma sorting, tentukan operasi utama, bentuk $T(n)$, dan Big-O. Hubungkan hasil teori tersebut dengan kecenderungan estimasi waktu pada nomor 9.

12. Jelaskan mengapa `print` tidak boleh berada di dalam fungsi yang sedang diukur dengan `timeit`, walaupun `print` justru sangat berguna pada versi trace untuk memahami proses algoritma.